Supplementary Information
JobMatch: An Agentic Multi-Role Platform for Explainable Job–Candidate Matching

Scope

This supplementary note preserves implementation and audit detail moved out of the main manuscript during page reduction. It does not introduce new experimental results.

Implementation Map

The main manuscript summarizes the runtime boundaries. Table 1 lists the concrete backend artifacts behind those boundaries.

Table 1: Backend implementation map.

Layer	Repository artifact	Responsibility
Gateway	`gateway/app.py`, `gateway/routes/*`	Builds the FastAPI app and exposes `/auth`, `/agents`, `/candidates`, `/jobs`, `/match/*`, `/similar`, `/feedback`, `/system`, and `/real-jobs`.
Container	`bootstrap.py`	Constructs agents, stores, event bus, seeded corpora, and optional live-job snapshot replacement.
Candidate agent	`agents/candidate_agent.py`, `hooks/parser.py`	Parses resumes, versions candidate snapshots, embeds canonical text, and upserts to `candidates_collection`.
Employer agent	`agents/employer_agent.py`	Registers job postings, embeds descriptions, upserts to `jobs_collection`, and supports `replace_corpus` for bulk refresh.
Matchmaking agent	`agents/matchmaking_agent.py`	Reads snapshots, runs retrieval/scoring, emits `MATCH_COMPLETED`, and never mutates profile stores.
Scoring core	`core/scoring.py`, `matchmaking_scoring.py`, `component_scores.py`	Implements semantic, skill, title, experience, compensation, remote, multimodal, and composite scores.
Optional rankers	`core/rrf.py`, `fusion.py`, `calibration.py`, `feedback_boost.py`, `cross_encoder_rerank.py`	Adds ensemble fusion, learned fusion, Platt calibration, user-feedback deltas, and cross-encoder reranking when enabled.
Explanations	`core/match_explanation.py`, `core/explain.py`	Produces structured fit signals, score breakdowns, and rule-based explanation bullets for portal cards and drawers.
Stores	`stores/factory.py`, `auth/store.py`, `feedback_store.py`, `candidate_activity_store.py`	Selects Chroma/Qdrant, persists roles and sessions, stores feedback, saved jobs, applications, and activity rows.

User-Facing Workflows

Table 2 lists the role-specific portal routes and API/agent paths summarized in the main implementation section.

Table 2: Primary user-facing workflows by role (K=10 composite matching unless noted).

Role	Portal route	User goal	API / agent path
Candidate	`/candidate/onboarding`	Upload resume, review extraction, create profile	`POST/candidates/upload-resume` → `PUT/candidates/me` → Candidate agent registers snapshot + embedding
Candidate	`/candidate/profile`	Edit skills, experience, pay, remote prefs	`GET/PUT/candidates/me`; quality via `POST/candidates/quality-check`
Candidate	`/candidate/matches`	View ranked jobs, filter, explain scores	`POST/match/candidate-to-jobs` (composite); profile readiness gate before search
Candidate	`/candidate/saved`	Saved jobs and applications	`GET/PUT/candidates/me/saved-jobs`; `POST/candidates/me/applications`
Employer	`/employer/jobs`	Import JD, post/edit/close roles	`POST/jobs/upload-description` or `/parse-description` → `POST/jobs` or `PUT/jobs/mine/{id}` → Employer agent
Employer	`/employer/matches`	Rank candidates for an owned job	`POST/match/job-to-candidates` (composite); job ownership required
Employer	`/employer/applications`	Review applicants per posting	`GET/jobs/mine/applications` (SQLite activity store)
Admin	`/admin/console`	Monitor agents, run matches, inspect fairness	`GET/agents/status`, `/agents/events/recent`, `/system/config`; optional `/match/*`, `/system/fairness`, `POST/real-jobs/sync` when live feed enabled

All authenticated routes use cookie sessions (`withCredentials`) and FastAPI `require_role` guards. Match endpoints invoke the Matchmaking agent read-only over snapshots.

Data Contracts

Table 3 lists the snapshot and response contracts used by the Matchmaking Agent and portals.

Runtime Defaults

Table 4 records the portal defaults, optional branches, and live-job gates used by the prototype.

Runtime Matching Pipeline

Figure 4 in the main manuscript details the matching pipeline: retrieval, composite scoring, feasibility constraints, optional fusion/calibration, and structured explanations consumed by portal cards and drawers.

Reproducibility Assets

Table 5 lists the frozen corpus, benchmark drivers, and verification paths.

Portal Screenshots

The screenshots in Figure 1 show the candidate, employer, and score-breakdown surfaces that consume the matching pipeline. They use the same demo corpus and portal-default composite strategy as the main manuscript.

Evaluation Pipeline

The frozen-corpus evaluation pipeline used to generate the main manuscript tables is described in Section 6 of the main manuscript: frozen demo corpus, benchmark drivers, metric aggregation, fairness audits, optional model training, regression gates, and paper tables.

Table 3: Primary data contracts used by the Matchmaking agent and portals.

Contract	Key fields	Where it is used
Candidate snapshot	• identity: `id`, `name` • profile: `skills`, experience, salary, remote preference • trace: `version`, text hash, `embedding`	Created by the Candidate agent after resume/profile updates; scored by the Matchmaking agent.
Job snapshot	• posting: title, description, required/preferred skills • fit constraints: experience, budget, remote policy • metadata: company, link, location, source, version, hash, embedding	Created by the Employer agent from portal jobs, snapshots, or live feeds; vector metadata supports filters and admin inspection.
Match result	• target: `target_id`, label, rank • scores: similarity, six components, `final_score` • context: skills, metadata, constraints, optional explanation	Returned to candidate and employer portals as the ranked item row.
Match explanation	• matched and missing skills • four fit signals • weighted score breakdown	Drives the explanation drawer and score-breakdown UI.
Match response	• `session_id`, direction, strategy • corpus counters and agent versions • results and optional rerank diagnostics	Wraps each matching call so tests and portals can inspect strategy and corpus context.

Table 4: Runtime and evaluation defaults.

Setting group	Default value	Notes
Embedding and vector store	• model: `all-MiniLM-L6-v2` • dimension: 384 • default store: `vector_store=chroma`	Qdrant is available through the same store interface; Chroma uses cosine space by default.
Portal match request	• `topK=10`; exhaustive retrieval • composite, cosine, Jaccard skills • rules explanations	Used by `DEFAULT_CANDIDATE_MATCH` and `DEFAULT_EMPLOYER_MATCH` on the demo corpus.
API/evaluation request	• `top_k=5` • `candidate_pool=120` • exhaustive retrieval	Offline evaluation and regression tests use K=5 unless a driver overrides `-top-k`.
Composite weights	• semantic 28%; skills 27% • title 10%; experience 15% • compensation 10%; remote 10%	Matches the portal-default composite scorer and ablation.
Fusion and calibration	• `Settings.rrf_k=60` • `fusion.json` • `calibration.json`: $a\approx 0.298$, $b\approx -2.116$	Available for evaluation/experiments; calibration is disabled in portal defaults.
Explanation labels	• Strong ≥ 0.85 • Good ≥ 0.65 • Moderate ≥ 0.45; else Weak	Rule bullets additionally use title and semantic narrative cutoffs.
Constraint and feedback branches	• experience-gap penalty: 0.65 • save $+0.04$; apply $+0.06$ • dismiss -0.06	Applied only when constraints or feedback boost are enabled.
Cross-encoder branch	• rerank pool: 20 • `enable_cross_encoder_rerank=false`	Disabled in production settings and portal defaults.
Live-job sync	• `REAL_JOBS_ENABLE=false` • page limit 100; timeout 30s • output: `data/jobs_live.json`	Daily batch pre-syncs only when live jobs are enabled.

Table 5: Reproducibility assets and verification paths.

Category	Files or commands	Purpose
Frozen corpus	• `data/cvs.json` • `data/jobs.json` • `data/eval_pairs.json`	Provides 30 resumes, 15 jobs, and 47 graded query–document relevance pairs.
Auxiliary assets	• fairness profiles and expected metrics • fusion and calibration models	Stores fairness probes, expected metrics, learned fusion weights, and Platt calibration parameters.
Research mirror	`data/research/*`	Mirrors the corpus and manifest used by paper-facing export scripts.
Benchmark drivers	• comparison, ablation, paper tables • significance, explainability, fairness	Recomputes ranking metrics, ablations, paper tables, significance tests, explainability checks, and fairness audit outputs.
Pipeline wrapper	`backend/scripts/run_research_pipeline.py`	Runs the orchestrated research pipeline with default `-top-k 5`.
Test runner	• `bash scripts/run_tests.sh` • backend: `pytest ../tests -q`	Runs Python unit/integration/benchmark tests plus frontend utility tests.
Regression gate	`tests/benchmarks/test_eval_regression.py`	Recomputes semantic and multimodal rankings and checks nDCG@5 against frozen expected metrics with ± 0.04 tolerance.
Live-job contract tests	• `test_real_jobs_sync.py` • `test_real_jobs_api.py`	Verifies normalization, pagination mock, snapshot I/O, status endpoint, and disabled-sync behavior.

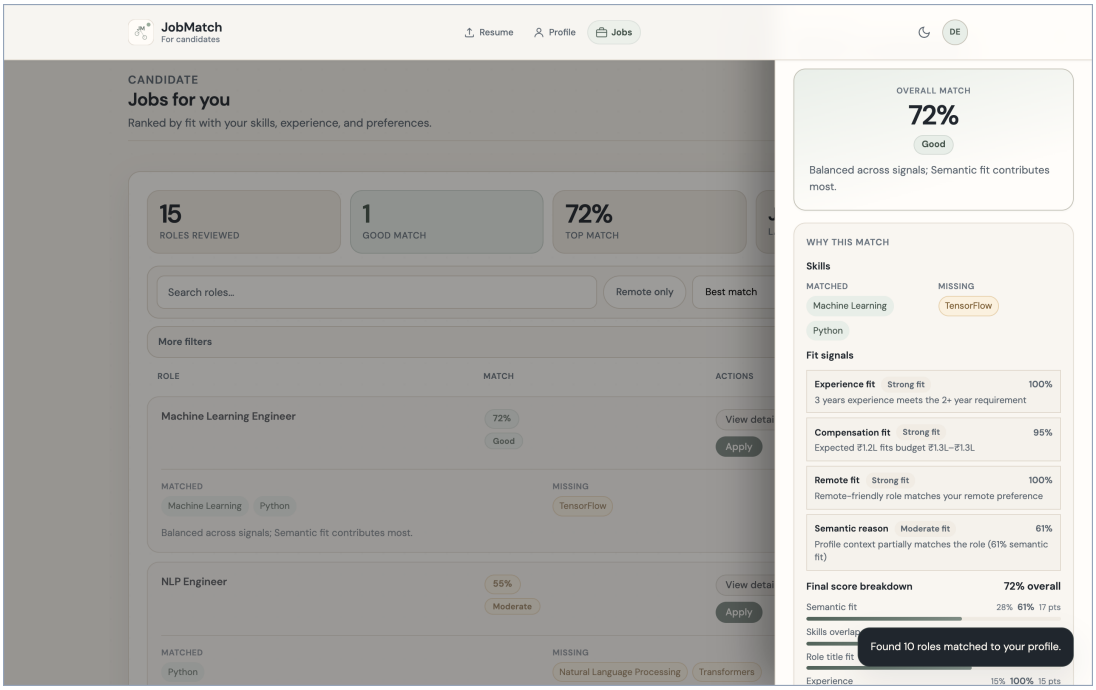

Figure 1: Portal screenshot: candidate job matches with explanation drawer.

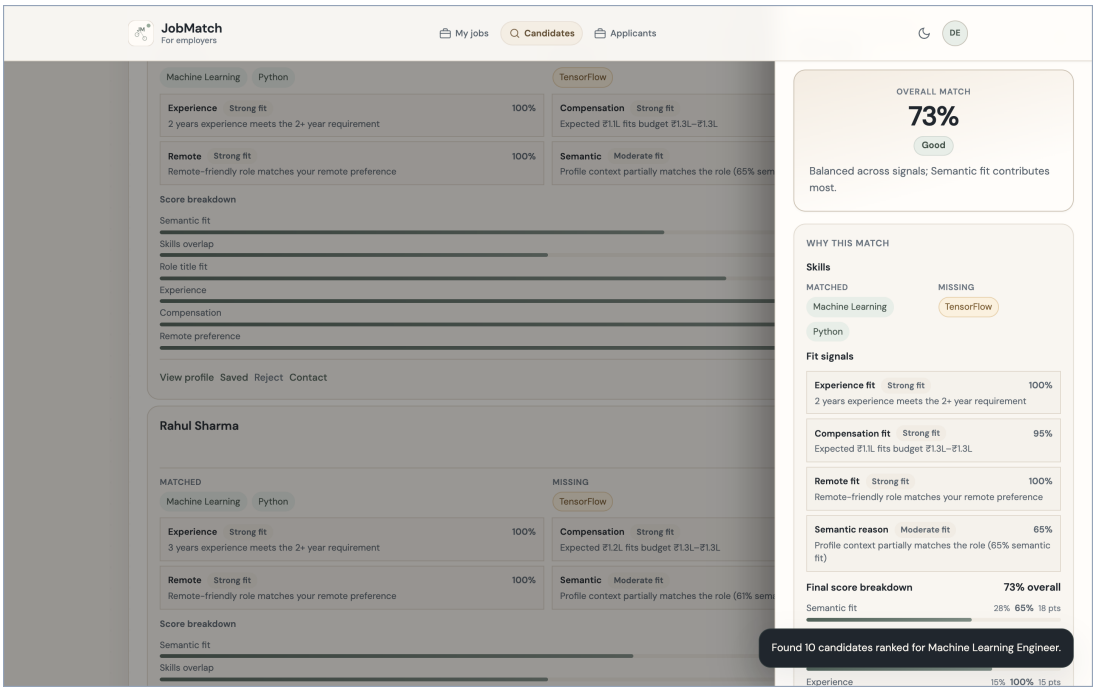

Figure 2: Portal screenshot: employer reverse candidate search for an owned posting.

Fusion and Model Parameters

Table 6 gives the full method comparison summarized in the main results section. Table 7 gives the fusion and constraint variants, and Table 8 records the fitted calibration and learned-fusion parameters.

Table 6: Retrieval method comparison (macro-averaged, K=5).

Method	Family	P@5	R@5	MRR	nDCG@5	MAP
Multimodal weighted	Embedding	0.287	0.933	0.961	0.924	0.867
RRF ensemble	Embedding	0.293	0.950	0.944	0.913	0.845
TF-IDF cosine	Lexical	0.293	0.950	0.918	0.898	0.842
BM25	Lexical	0.307	0.983	0.912	0.894	0.838
Semantic cosine	Embedding	0.267	0.867	0.931	0.878	0.810
Semantic euclidean	Embedding	0.267	0.867	0.931	0.878	0.810
Soft skill embed	Embedding	0.280	0.900	0.911	0.869	0.829
Exact skill overlap	Lexical	0.233	0.733	0.816	0.748	0.681
Skills Jaccard	Embedding	0.233	0.733	0.816	0.748	0.681

Source: research pipeline run `2026-05-27T150000Z/comparison/comparison_summary.json`. Exhaustive ranking over all 15 jobs per query; detailed latency values are provided in the supplementary information.

Ablation and Latency Details

Tables 9 and 10 provide the detailed ablation and latency rows summarized in the main results section.

Hard-Negative, Fairness, and Explanation Audits

Tables 11, 12, and 13 provide the detailed audit outputs summarized in the main manuscript.

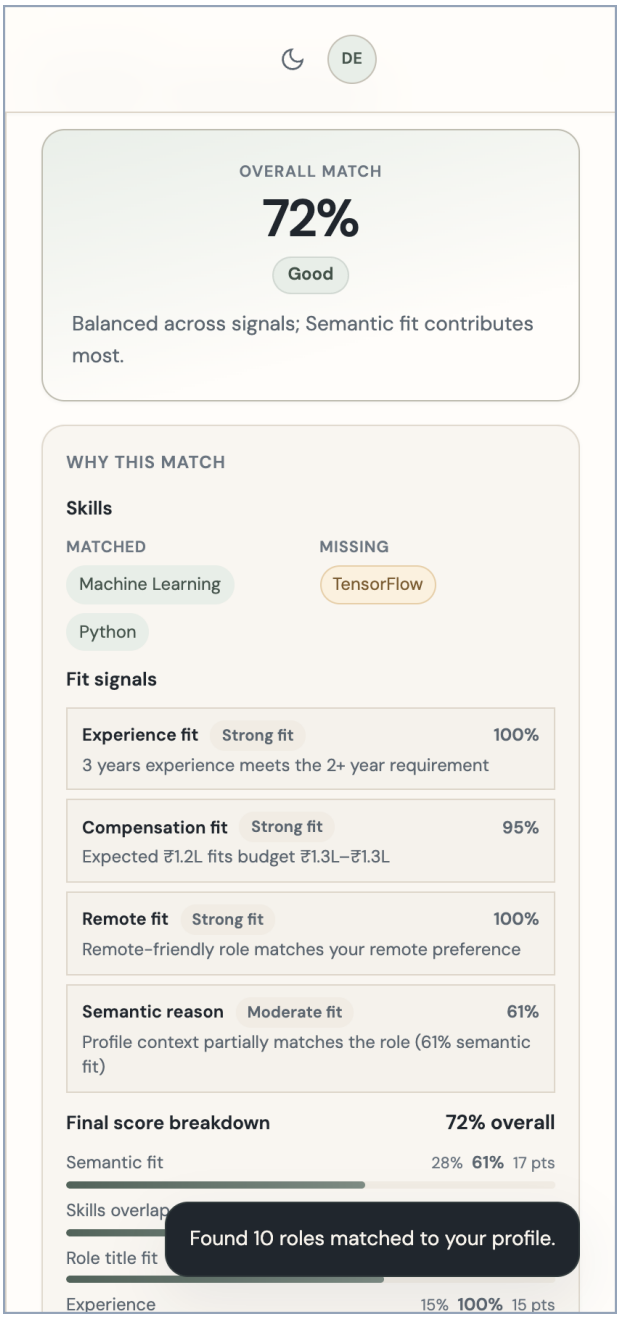

Figure 3: Portal screenshot: composite score breakdown and skill alignment.

Table 7: Fusion and constraint variants (K=5).

Method	P@5	R@5	nDCG@5
Learned fusion (LR)	0.307	0.983	0.968
Fixed multimodal (α=0.7)	0.287	0.933	0.924
Hierarchical skills	0.287	0.933	0.924
Multimodal + constraints	0.300	0.950	0.908
Semantic cosine	0.267	0.867	0.878

Source: `backend/benchmark_outputs/table11_fusion.json` (`benchmarks/table11_fusion.py`).

Table 8: Fitted calibration and fusion model parameters (offline training).

Component	Value
Platt calibrator (`data/models/calibration.json`)	
a (scale)	0.298
b (shift)	-2.116
Learned fusion LR (`data/models/fusion.json`)	
Top feature weights	• experience 0.860; remote 0.853; salary 0.849 • semantic 0.537; taxonomy 0.509; skills 0.204 • must-have coverage 0.311; bias 1.529

Optional paths only: `use_calibration` or `fusion_mode=learned`. Platt calibration uses 450 composite-scored pairs and reduces ECE from 0.40 to 0.032 under 5-fold cross-validation (Brier 0.093); monotonic calibration leaves ranking order and nDCG@5 unchanged. Learned fusion is exploratory.

Table 9: Ablation over composite matching components (K=5).

Variant	Category	P@5	R@5	MRR	nDCG@5	MAP
Full composite	Full	0.293	0.933	0.972	0.949	0.921
Semantic + skills	Partial	0.287	0.933	0.961	0.917	0.867
Semantic + skills + exp.	Partial	0.287	0.933	0.961	0.916	0.865
Semantic only	Single	0.267	0.867	0.931	0.878	0.810
Skills only	Single	0.233	0.733	0.816	0.748	0.681
Compensation only	Single	0.187	0.567	0.457	0.393	0.387
Location only	Single	0.167	0.467	0.388	0.335	0.328
Experience only	Single	0.160	0.467	0.384	0.326	0.314
Component RRF	Ensemble	0.233	0.717	0.570	0.564	0.497

Source: `benchmarks/ablation.py` (portal-default run). Full-composite weights are semantic 28%, skills 27%, title 10%, experience 15%, compensation 10%, remote 10%; title overlap enters through the composite scorer. Single-component and partial rows renormalize the listed weights.

Table 10: Quality–latency trade-off (exhaustive evaluation, K=5).

Method	nDCG@5	Latency (ms/q)	Notes
Exact skill overlap	0.748	0.03	Lexical
Skills Jaccard	0.748	0.03	Set overlap
TF-IDF cosine	0.898	0.07	Lexical
BM25	0.894	0.11	Lexical
Semantic euclidean	0.878	0.21	Embedding
Semantic cosine	0.878	0.23	Embedding
Multimodal weighted	0.924	0.26	α=0.7, Jaccard skills
RRF ensemble	0.913	0.95	Four ranked lists
Semantic (ANN, pool 10)	0.878	1.9	Chroma shortlist
Multimodal (ANN, pool 10)	0.919	1.9	Chroma shortlist, α=0.7
Soft skill embed	0.869	15.53	Per-skill embedding
Composite (bi-encoder)	0.949	0.41	Ablation full composite
Composite + cross-encoder	0.834	141.7	Pool=20; ΔnDCG=$-$0.108

Latency from `comparison_summary.json`, `cross_encoder_report.json`, and `phase11_summary.csv` (ANN, pool=10, Chroma). Exhaustive scoring and ANN shortlist timings are measured on the 15-job pool, so per-query latencies are small and noisy at this scale.

Table 11: Hard-negative mining summary (multimodal top-10 pool, K=5 negatives per query).

Statistic	Value
Queries mined	30
Negatives per query (target / observed)	5 / 5
Total hard-negative pairs	150
Negatives with graded relevance > 0	0
Negatives overlapping declared relevant set	0

Source: `backend/benchmark_outputs/hard_negatives.json`. Mined with multimodal weighted scoring (α=0.7) over 15 jobs; negatives are high-scoring non-labeled jobs, not confirmed irrelevant documents.

Table 12: Synthetic counterfactual fairness audit (10 pairs, composite ranking).

Pair	Field	Top-1	Top-5	Rank Δ	Score Δ	Flag
Name gender 1	name	Yes	5/5	2	0.0067	No
Name gender 2	name	Yes	5/5	1	0.0050	Yes
Ethnicity 1	name	Yes	5/5	0	0.0110	Yes
Ethnicity 2	name	Yes	4/5	3	0.0167	Yes
Nationality 1	summary	Yes	5/5	1	0.0061	Yes
Nationality 2	summary	Yes	5/5	1	0.0077	No
Hometown 1	hometown	Yes	5/5	1	0.0082	Yes
Hometown 2	hometown	Yes	4/5	2	0.0100	Yes
Pronouns 1	pronouns	Yes	5/5	0	0.0029	No
Email domain 1	email	Yes	5/5	1	0.0057	Yes

Source: `benchmarks/fairness_audit.py`; fabricated profiles only. Seven of 10 pairs are flagged, all with the top-1 job stable; maximum score shift is 0.017. Proxy-group DIR from `fairness_eval.py`: experience tiers 0.82, remote preference 0.75 (1.0 indicates parity).

Table 13: Offline explanation-quality audit (top-5 composite matches, K=5).

Explainer	Faithful	Spec.	Skill	No halluc.	Comp.	Consist.
Rule-based (default)	0.745	0.627	25.3%	98.0%	100%	1.00
Grounded template	0.747	0.633	26.7%	97.3%	100%	1.00

Source: `benchmarks/explainability_eval.py`; 300 explanations (30 candidates $\times$ top-5 $\times$ 2 modes). Faithfulness is the fraction of three hard checks passed; consistency is bullet-set Jaccard across 10 synthetic profile pairs. Both modes align component claims and are consistent across variants, but explicit matched/missing skills appear in only $\sim$26% of bullets, driving the flagged rate ($\sim$77%).

Primary benchmark artifacts

Artifact	Path (repository root)
Paper progression summary	`data/expected/paper_progression_summary.json`
Fusion ablation	`backend/benchmark_outputs/table11_fusion.json`
Composite ablation	`docs/research/evaluation/artifacts/runs/2026-05-27T150000Z/ablation/ablation_summary.json`
Method comparison	`docs/research/evaluation/artifacts/runs/2026-05-27T150000Z/comparison/comparison_summary.json`
Cross-encoder report	`docs/research/evaluation/artifacts/runs/2026-05-27T150000Z/cross_encoder/cross_encoder_report.json`
Hard negatives	`backend/benchmark_outputs/hard_negatives.json`
Fairness audit table	`docs/research/evaluation/paper_tables/table4_fairness.csv`
Eval corpus	`data/cvs.json`, `data/jobs.json`, `data/eval_pairs.json`

Regeneration commands

```
python backend/scripts/run_research_pipeline.py
python -m benchmarks.paper_progression
python -m pytest tests/benchmarks/test_eval_regression.py -q
```

Regression gate

Committed progression baselines are gated by `tests/benchmarks/test_eval_regression.py` with ±0.04 nDCG tolerance on nDCG@5.

Aligned with manuscript declarations (May 2026).